

## Avoiding Argument Errors

When you start to use functions, don't be surprised if you encounter errors about unmatched arguments. Unmatched arguments occur when you provide fewer or more arguments than a function needs to do its work. For example, here's what happens if we try to call `describe_pet()` with no arguments:

---

```
def describe_pet(animal_type, pet_name):  
    """Display information about a pet."""  
    print(f"\nI have a {animal_type}.")  
    print(f"My {animal_type}'s name is {pet_name.title()}")  
  
describe_pet()
```

---

Python recognizes that some information is missing from the function call, and the traceback tells us that:

---

```
Traceback (most recent call last):  
❶ File "pets.py", line 6, in <module>  
❷   describe_pet()  
   ^^^^^^^^^^^^^^^^^  
❸ TypeError: describe_pet() missing 2 required positional arguments:  
   'animal_type' and 'pet_name'
```

---

The traceback first tells us the location of the problem ❶, allowing us to look back and see that something went wrong in our function call. Next, the offending function call is written out for us to see ❷. Last, the traceback tells us the call is missing two arguments and reports the names of the missing arguments ❸. If this function were in a separate file, we could probably rewrite the call correctly without having to open that file and read the function code.

Python is helpful in that it reads the function's code for us and tells us the names of the arguments we need to provide. This is another motivation for giving your variables and functions descriptive names. If you do, Python's error messages will be more useful to you and anyone else who might use your code.

If you provide too many arguments, you should get a similar traceback that can help you correctly match your function call to the function definition.

### TRY IT YOURSELF

**8-3. T-Shirt:** Write a function called `make_shirt()` that accepts a size and the text of a message that should be printed on the shirt. The function should print a sentence summarizing the size of the shirt and the message printed on it.

Call the function once using positional arguments to make a shirt. Call the function a second time using keyword arguments.